# Signed Positional Prüfer Sequences: A Linear-Time, Pure Integer-Array Bijection for Rooted, Directed, and Ordered Trees

Anandhu P Shaji, Abishek vs, Gisha Marin Jose
College of Engineering, Munnar
APJ Abdul Kalam Technological University, Thiruvananthapuram, India

**Abstract**

The classical Prüfer sequence establishes an elegant bijection between labeled, unrooted trees and integer sequences, but it inherently omits root identity, edge directionality, and spatial child ordering. Existing adaptations for ordered trees typically achieve lossless reconstruction by augmenting the sequence with external metadata arrays, sacrificing the spatial contiguity of a pure one-dimensional encoding.

We present the Signed Positional Prüfer Sequence (SPPS), a bijective encoding that achieves complete lossless reconstruction of rooted, directed, and strictly ordered trees using only a single integer array of length $n - 1$. Our approach combines a virtual root injection strategy with a mixed-radix packing technique that deterministically encapsulates topological structure, edge orientation, and exact left-to-right spatial ordering into an expanded integer domain.

A sliding-pointer algorithmic realization is presented that achieves exact lossless reconstruction in strictly $\mathcal{O}(n)$ linear time and $\mathcal{O}(n)$ space. In empirical evaluations, a 5.3× encoding speedup over Google Protocol Buffers is achieved. At scale, the peak spatial footprint is reduced by 50%, occupying 172.93 MB compared to 346.83 MB for an Abstract Syntax Tree of 2.3 million nodes. A fixed-width serialization payload of exactly 8.00 bytes/node is maintained across all scales, trading wire compactness against variable-length formats for constant-time decoding and hardware-aligned cache access. Furthermore, by restructuring tree topology into cache-sequential integer arrays and substantially reducing pointer-chasing during the serialization phase, traversal latency variance is restricted to 4.40%. An efficient, single-allocation contiguous serialization format is thus provided, which is well-suited for archiving immutable computational topologies, such as frozen compiler Abstract Syntax Trees (ASTs) and finalized Neural Architecture Search (NAS) structures.

**Keywords:** Prüfer Sequences, Tree Bijections, Data Serialization, Graph Algorithms, Tree Encoding, Cache Locality.

## 1 Introduction

The serialization of hierarchical data structures is fundamental to computer science, with applications spanning compiler design (Abstract Syntax Trees), database systems (XML document indexing), computational biology, and artificial intelligence (Neural Architecture Search topologies). As computational graphs scale to millions of nodes, the need for efficient, mathematically rigorous, and lossless tree encodings remains critical.

Historically, the Prüfer sequence [1] has served as the canonical tree encoding for combinatorial mathematics. Introduced to provide an elegant proof of Cayley's formula [2], the classical algorithm establishes a bijection between labeled, unrooted trees of $n$ vertices and integer sequences of length $n - 2$. However, when applied to real computational structures—which are inherently rooted, directed, and spatially ordered—the classical Prüfer sequence

omits critical structural information: it drops root identity, ignores directed edge flow, and alters the left-to-right ordering of sibling nodes due to its reliance on numerical sorting during the leaf-removal process.

### 1.1 Limitations of Existing Approaches

To address these limitations, prior work has explored two primary directions:

**State-Augmented Prüfer Variants.** PRIX [5] introduced a Prüfer-based encoding for XML twig query processing. Each document is transformed into a Labeled Prüfer Sequence (LPS) of element tags and a Numbered Prüfer Sequence (NPS) of postorder node identifiers, both stored in $B^+$-tree indexes to support subsequence matching against query twigs. While functionally well-suited for active XML database querying, this approach maintains multiple parallel sequence arrays per document alongside explicit leaf-node lists—a layout designed for query throughput rather than space-minimal archival serialization. Subsequent Prüfer-based XML indexing systems extended this direction with additional structural metadata; all remain oriented toward active query processing rather than archival serialization, and are thus architecturally distinct from SPPS.

**Succinct Tree Representations.** In theoretical computer science, succinct data structures [7, 8] achieve information-theoretic space optimality, encoding an ordered tree of $n$ nodes in $2n + o(n)$ bits. While asymptotically optimal, these bit-level encodings are complex to implement, require specialized rank/select operations, and are generally incompatible with integer-tokenized environments such as compiler intermediate representations and neural network architectures.

### 1.2 Our Contribution: Pure Integer-Array Bijection

In this paper, we demonstrate that external metadata augmentation is mathematically unnecessary. We introduce the Signed Positional Prüfer Sequence (SPPS), which expands the information capacity of the sequence alphabet itself to natively absorb all structural state without requiring auxiliary arrays. While classical Prüfer encoding and decoding algorithms already achieve $\mathcal{O}(n)$ time complexity, our core contribution is representational novelty: establishing a pure one-dimensional integer encoding that preserves all structural state without external metadata arrays.

SPPS is built on four key innovations. **Virtual Root Injection** introduces a virtual node technique that mathematically forces the Prüfer sequence to deterministically preserve the root identity of any tree without a separate state variable. **Bijective Radix Packing** defines a deterministic mapping function $\omega = d \cdot (P \cdot N + k)$ that simultaneously encapsulates the parent ID ($P$), edge direction ($d \in \{-1, +1\}$), and strictly ordered spatial rank ($k$) into a single integer via mixed-radix packing. **Pure 1D Bijection** formally proves that SPPS achieves exact lossless reconstruction of rooted, directed, and canonically ordered trees using only a single integer array of length $n - 1$. Finally, **Cache-Efficient Sliding-Pointer Realization** provides an encoding and decoding implementation that runs in strictly $\mathcal{O}(n)$ linear time by bypassing comparison-based sorting and relying instead on $\mathcal{O}(1)$ direct memory addressing, achieving a contiguous memory layout that minimizes cache-line misses during serialization.

### 1.3 Related Work

**Classical Prüfer Sequences and Tree Bijections:** The Prüfer sequence [1] provides a bijection between labeled, unrooted trees on $n$ vertices and integer sequences of length $n - 2$. The classical encoding algorithm iteratively removes the leaf with the minimum label, recording its unique neighbor. This process inherently canonicalizes the tree structure, discarding

the original spatial ordering of siblings. Furthermore, the resulting sequence encodes an unrooted topology. Extensions by Neville [3] and Moon [4] explored rooted tree variants but did not address edge directionality or spatial ordering.

**Prüfer-Based XML and Hierarchical Indexing:** PRIX [5] introduced a Prüfer-based encoding for XML twig pattern matching. Each XML document is transformed into a Labeled Prüfer Sequence of element tags and a Numbered Prüfer Sequence of postorder node identifiers; twig queries are similarly encoded and matched via subsequence inclusion against the indexed sequences. The goal of PRIX is query throughput rather than archival serialization: it stores multiple parallel sequence arrays per document alongside explicit leaf-node lists in $B^+$-tree indexes, a layout oriented toward fast query processing. FIST [6] further optimizes twig-pattern filtering on top of this framework but similarly relies on multi-array structural metadata oriented toward query processing rather than archival storage.

### 1.4 The Serialization Bottleneck in ASTs and NAS

Standard text-based formats like JSON and XML dominate data transmission but introduce unacceptable bottlenecks in high-throughput computational environments. A standard JSON representation of an Abstract Syntax Tree (AST) incurs a spatial overhead approaching $\approx 10n \log_2 n$ bits due to syntax characters and redundant keys. While binary formats like Protocol Buffers are more compact, they still require costly serialization/deserialization passes to instantiate in-memory pointer graphs.

SPPS provides a highly contiguous, single-allocation serialization alternative: an AST of $n$ nodes natively packs into a flat integer array. While instantiating the navigable pointer graph still strictly requires an $\mathcal{O}(n)$ unpacking phase, this format allows distributed compiler instances or Neural Architecture Search (NAS) workers to write entire immutable graph topologies directly to disk via memory-mapped I/O, bypassing the complex pointer-chasing serialization bottlenecks typical of nested tree structures.

## 2 Preliminaries

### 2.1 Tree Definitions and Problem Formulation

**Definition 2.1** (Rooted, Directed, Ordered Tree)**.** A rooted, directed, ordered tree $T$ is a tuple $(V, E, r, <)$ where $V = \{1, 2, \ldots, n\}$ is a set of $n$ uniquely labeled, contiguous integer vertices (if an external system provides non-contiguous, sparse, or string-based identifiers, an initial $\mathcal{O}(n)$ hashing and relabeling phase is strictly required to normalize the vertex set prior to encoding); $E \subset V \times V$ is a set of $n-1$ directed edges such that the underlying undirected graph is connected and acyclic, with every vertex $v \neq r$ having exactly one incoming edge (unique parent); $r \in V$ is the designated root vertex; and for each vertex $v \in V$, the relation $<$ is a strict total order on $\text{children}(v) = \{u : (v, u) \in E\}$ for all $v$, defining a left-to-right spatial ordering. For each child $c$ of parent $p$, we define its spatial rank $k_c \in \{0, 1, \ldots, |\text{children}(p)|-1\}$ as the 0-indexed position of $c$ in the left-to-right ordering of $\text{children}(p)$ under $<$.

**Definition 2.2** (Lossless Tree Encoding)**.** An encoding scheme $(f, g)$ for rooted, directed, ordered trees is lossless if $f : \mathcal{T} \to S$ maps a tree $T$ to a sequence $S$ of integers, $g : S \to T'$ reconstructs a tree $T'$ from $S$, and the round-trip condition $g(f(T)) = T$ holds up to isomorphism, meaning the reconstructed tree $T'$ preserves topology, root identity, edge directionality, and spatial ordering.

**Formal Bijection Definition:** SPPS establishes a strict bijection between the set of rooted, directed, ordered trees and a highly constrained codomain of integer sequences, where every integer strictly obeys the bounds of the mixed-radix packing $\omega = d \cdot (P \cdot N + k)$.

**Goal:** Design $(f, g)$ such that $S$ is a single integer array of length $n-1$ achieving strictly $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space complexity.

### 2.2 Core Idea: Mixed-Radix Packing

To eliminate external metadata, SPPS leverages integer word-size scaling. Instead of encoding only the parent ID $P \in \{1, \ldots, n\}$, we construct a composite value using mixed-radix packing:

$$\omega = d \cdot (P \cdot N + k) \tag{1}$$

The parameters are: $d \in \{-1, +1\}$ is the direction bit encoded as the sign of $\omega$, where $+1$ indicates an edge directed from parent to child (standard tree orientation) and $-1$ indicates an edge directed from child to parent; $P \in \{1, \ldots, n+1\}$ is the parent vertex label, augmented to include the virtual node; $k \in \{0, 1, \ldots, N-1\}$ is the spatial rank of the child among its parent's children (0-indexed, left-to-right); and $N = n + 2$ is the scaling constant chosen to prevent domain overlaps with the virtual node $n + 1$.

Intuitively, we partition the integer domain into non-overlapping "slots," one per parent ID. Each parent $P$ occupies a contiguous block of $N$ consecutive integers, starting at $P \cdot N$. Within this block, we encode the child's spatial rank $k \in \{0, \ldots, N-1\}$ by adding $k$ to the base address $P \cdot N$. The direction bit $d$ is encoded via the sign of $\omega$. Because $N = n + 2 > \max(k) + 1$, different parents' blocks never overlap, ensuring strict injectivity.

Because $0 \leq k < N$, the value $P \cdot N + k = (n+1)(n+2) + (n+1) < (n+2)^2$ is well-separated from adjacent parent domains.

### 2.3 Virtual Root Injection

To encode root identity natively, we augment the tree with a virtual node:

**Definition 2.3** (Augmented Tree)**.** Given a tree $T = (V, E, r, <)$ with $V = \{1, \ldots, n\}$ and root $r$, define the augmented tree $T' = (V', E', r, <')$ where $V' = V \cup \{n+1\}$ adds a virtual node with label $n + 1$; $E' = E \cup \{(r, n+1)\}$ adds a directed edge oriented from the original root $r$ to the virtual node $n + 1$; and $<'$ extends the relation $<$ by appending $n + 1$ as an additional child of $r$, assigned the next available spatial rank (i.e., rank $|\text{children}(r)|$ in the original tree).

**Key Property:** During Prüfer encoding, the algorithm iteratively removes the leaf with the minimum label. Because $n+1 > v$ for all $v \in V$, the virtual node $n+1$ is never the minimum leaf in any intermediate state. Therefore, $n+1$ survives until the final step, when exactly two nodes remain. At turnaround, the virtual node's unique neighbor is deterministically the original root $r$.

### 2.4 Theoretical Analysis

**Definition 2.4** (Valid SPPS Codomain)**.** Let $\mathcal{T}$ be the set of all rooted, directed, ordered trees on $n$ labeled vertices. Let $\mathcal{S}_{\text{SPPS}}$ be the set of integer sequences of length $n-1$ where each element $\omega_i = d_i \cdot (P_i \cdot N + k_i)$ satisfies $P_i \leq n + 1$ and $k_i < N$, and the parent sequence $\{P_1, \ldots, P_{n-1}\}$ forms a valid unrooted tree topology.

**Lemma 2.1** (Virtual Node Survival under Sliding Pointer)**.** In the augmented tree $T' = (V', E', r, <')$ with virtual node $n + 1$, the root $r$ is deterministically the final leaf processed during encoding, and $n + 1$ is deterministically its parent in the final recorded token, guaranteeing root preservation.

*Proof.* Let $V' = \{1, 2, \ldots, n, n+1\}$. In the sliding-pointer algorithm, a node is selected as the current leaf either via the monotonically increasing scanner ptr or the $\mathcal{O}(1)$ shortcut condition $P < \text{ptr}$.

The shortcut condition mathematically simulates a minimum-priority queue without comparison-based sorting: it only triggers when a newly exposed leaf ($P$) has a label strictly less than the current scanner position. Because the scanner strictly increments and $v_{\text{virt}} = n + 1$ is the maximum label in $V'$, the virtual node can never satisfy $P < \text{ptr}$ and is therefore never selected as a leaf via the shortcut. Nor is it ever the minimum unprocessed leaf reached by the scanner, since all labels $1, \ldots, n$ are smaller and are consumed first. Consequently, $n + 1$ retains degree 1 throughout all $n$ iterations. In the final ($n$-th) iteration, the only remaining leaf is the root $r$, whose unique remaining neighbor in $T'$ is $n + 1$. The token appended in this iteration therefore has $P = n+1$, deterministically encoding the root identity. This final token is subsequently discarded (Algorithm 1, final step before **return**), leaving a sequence of length $n - 1$ that encodes the original tree. □

**Lemma 2.2** (Bijectivity of the Mixed-Radix Packing)**.** For fixed $N = n + 2$, the mapping $f(P, k, d) = d \cdot (P \cdot N + k)$ where $P \in \{1, \ldots, n + 1\}$, $k \in \{0, \ldots, N - 1\}$, and $d \in \{-1, +1\}$ is injective: distinct triples $(P, k, d)$ map to distinct integers $\omega$.

*Proof.* Since $P \geq 1$, $N > 0$, and $k \geq 0$, we have $P \cdot N + k \geq N > 0$. Thus, the sign of $\omega$ deterministically recovers $d = \text{sign}(\omega)$. Taking the absolute value yields $|\omega| = P \cdot N + k$. Since $0 \leq k < N$, we have $P = \lfloor |\omega|/N \rfloor$ and $k = |\omega| \pmod N$. Thus, distinct triples $(P, k, d)$ deterministically map to distinct integers $\omega$, confirming a strict 1:1 mapping. □

**Lemma 2.3** ($\mathcal{O}(1)$ Per-Child Spatial Rank Placement in $\mathcal{O}(n)$ Total)**.** For each parent $P_i$, let its children be strictly ordered left-to-right. During decoding, each child is placed into its exact spatial position in $\mathcal{O}(1)$ time via direct array addressing, restoring the original spatial ordering of all $n - 1$ children in $\mathcal{O}(n)$ total time without comparison-based sorting.

*Proof.* The invertible mixed-radix packing guarantees that the exact spatial rank $k_i \in \{0, \ldots, m-1\}$ of every child is intrinsically bound to the sequence token $\omega_i$. During decoding, $k_i$ is deterministically extracted via $k_i = |\omega_i| \pmod N$. Because the algorithm pre-computes the exact out-degree of every parent $P_i$ during the initial sequential scan, it allocates an exact-fit spatial map $M[P_i]$. By executing the direct memory assignment $M[P_i][k_i] = \text{leaf}$, the child is locked into its absolute left-to-right rank natively. This guarantees the strict total order is perfectly restored in $\mathcal{O}(1)$ time. □

**Lemma 2.4** (Totality and Surjectivity)**.** The algorithm defines a total function $f : \mathcal{T} \to \mathcal{S}_{\text{SPPS}}$, and the mapping onto $\mathcal{S}_{\text{SPPS}}$ is surjective.

*Proof.* **Part 1: Totality (Well-definedness)** Given any valid tree $T \in \mathcal{T}$, the virtual root injection (Definition 2.3) guarantees an augmented tree $T'$ with exactly $n + 1$ vertices and $n$ edges. The sliding-pointer algorithm removes exactly one leaf per iteration. Because $T'$ is a finite, connected, acyclic graph, every node (except the final two) will eventually reach a degree of 1 and be processed. This deterministically generates exactly $n - 1$ parent-child token triples $(P_i, k_i, d_i)$. By the fundamental properties of Prüfer sequences, the resulting parent sequence $\{P_1, \ldots, P_{n-1}\}$ inherently forms a valid unrooted topology. By Lemma 2.2, each triple natively packs into a unique composite integer $\omega_i$. Therefore, the output sequence $S$ strictly satisfies the bounds of $\mathcal{S}_{\text{SPPS}}$.

**Part 2: Surjectivity (Onto)** We construct an explicit left inverse $g : \mathcal{S}_{\text{SPPS}} \to \mathcal{T}$ and show $g(f(T)) = T$ for all $T \in \mathcal{T}$. Given any sequence $S \in \mathcal{S}_{\text{SPPS}}$, Algorithm 2 deterministically reconstructs a rooted, directed, ordered tree as follows: (i) each token $\omega_i$ is unpacked via $P_i = \lfloor |\omega_i|/N \rfloor$, $k_i = |\omega_i| \bmod N$, $d_i = \text{sign}(\omega_i)$, recovering the parent, spatial rank, and edge direction; (ii) the parent sequence $\{P_i\}$ forms a valid unrooted topology by Definition

2.4; (iii) the sliding-pointer pairing reconstructs directed edges; (iv) the virtual node $n+1$ deterministically identifies the root; and (v) spatial ranks $k_i$ restore the strict total child ordering. By construction, $g(f(T))$ reproduces exactly the topology, root, edge directions, and spatial ordering of $T$. Since $g \circ f = \mathrm{id}_{\mathcal{T}}$, $f$ is injective (Part 1 established totality and injectivity), and $g$ is a total function on $\mathcal{S}_{\mathrm{SPPS}}$, it follows that every element of $\mathcal{S}_{\mathrm{SPPS}}$ is the image of some $T \in \mathcal{T}$ under $f$. Therefore $f$ is surjective onto $\mathcal{S}_{\mathrm{SPPS}}$. □

**Theorem 2.1** (Lossless Reconstruction & Bijectivity)**.** For any rooted, directed, ordered tree $T = (V, E, r, <)$ with $|V| = n$, the SPPS encoding establishes a strict bijection, and the algorithms satisfy $\mathrm{Decode}(\mathrm{Encode}(T)) = T$.

*Proof.* **Topology and Vertex Set:** The parent sequence uniquely determines the topology of the augmented tree $T'$ as an unrooted tree on $n+1$ vertices. The decoding phase reconstructs this topology by pairing each leaf with its parent. Since the virtual node $n+1$ is subsequently removed, the resulting topology matches the original tree.

**Root Identity:** By Lemma 2.1, the final token appended during encoding has $P = n+1$, deterministically identifying the root $r$ as the last leaf processed. Decoding identifies $r$ as the vertex whose final parent is $n+1$.

**Edge Directionality:** The sign $d_i = \mathrm{sign}(\omega_i)$ is recovered and applied.

**Spatial Ordering:** The encoding phase records rank $k_i$. By Lemma 2.3, placing the children via $k_i$ natively restores the original order. □

**Theorem 2.2** (Strict Linear Time and Space Complexity)**.** The SPPS encoding and decoding algorithms operate in strictly $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space.

*Proof (Time).* The complete encoding pipeline consists of two sequential phases within Algorithm 1.

**Phase 1 (Initialization, Algorithm 1 lines 1–8):** The edge-count check is $\mathcal{O}(n)$. The linear scan over vertices $u = 1$ to $n$ processes each child exactly once; since the total number of children across all $n$ vertices equals $n-1$, the ChildRank assignments, parent array population, and neighbor-sum accumulation each cost $\mathcal{O}(n)$ in total. Phase 1 is therefore strictly $\mathcal{O}(n)$.

**Phase 2 (Sliding-pointer encoding, Algorithm 1 lines 9–29):** We use amortized analysis with the potential function $\Phi = \mathrm{ptr}$. Initially $\mathrm{ptr} = 1$. Each iteration performs $\mathcal{O}(1)$ work: one neighbor-sum lookup ($P \leftarrow \mathrm{nbr}[\mathrm{leaf}]$), one array write, one degree decrement, one neighbor-sum decrement, and either the $\mathcal{O}(1)$ shortcut or one ptr advance. Since ptr is monotonically non-decreasing and bounded by $n+1$, the total number of ptr increments across all $n$ iterations is at most $n$. The amortized cost per iteration is therefore $\mathcal{O}(1)$, and Phase 2 costs strictly $\mathcal{O}(n)$ for $n$ iterations.

Summing two $\mathcal{O}(n)$ phases gives $\mathcal{O}(n)$ total encoding time. Algorithm 2 (decode) follows an identical sliding-pointer structure with the same monotone bounded ptr argument, yielding $\mathcal{O}(n)$ decoding time. □

*Proof (Space).* Sequence $S$ contains $n-1$ signed 64-bit integers, yielding an exact wire-format payload of $8(n-1) \approx 8n$ bytes. The information-theoretic content of each packed integer $\omega = d \cdot (P \cdot N + k)$ is at most $\log_2(2 \cdot (n+1)(n+2)) \approx 2\log_2 n$ bits, so the theoretical minimum payload is approximately $2n \log_2 n$ bits. The fixed 64-bit representation incurs a modest overhead over this minimum for practical $n$, but provides hardware-aligned, cache-friendly access without variable-width encoding. The auxiliary structures allocated during encoding—the degree array $D$, the ChildRank array, the parent array, and the nbr neighbor-sum array—each have length $n+2$ and occupy $\mathcal{O}(n)$ space. No recursion stack is required. The overall spatial complexity is therefore strictly $\mathcal{O}(n)$. □

**Failure-Case Analysis and Algorithmic Guardrails:** SPPS is defined over valid rooted, directed, ordered trees. Algorithm 1 includes an edge-count check ($|E| = n - 1$) as a necessary precondition. For production deployments receiving externally sourced graphs, a full connectivity check should additionally be performed before encoding commences; the sliding-pointer loop is bounded ($\text{ptr} \leq n+1$) and will terminate regardless, but will produce a meaningless sequence if the input is not a tree. The benchmark implementation presented in Section 3 omits runtime validation because all inputs—the synthetic datasets and the Django AST—are structurally verified prior to execution; adding validation to a benchmarked hot path would conflate input-preparation cost with encoding cost.

**Hardware Bounds:** On standard 64-bit signed integer architectures, preventing structural overflow of the composite integer ($\omega \approx n^2 \leq 2^{63} - 1$) strictly bounds the maximum processable tree capacity to $n \leq \lfloor 2^{31.5} \rfloor$ nodes.

# 3 Algorithm

## 3.1 Input Representation and Contiguous Memory Layout

To mathematically guarantee strictly $\mathcal{O}(n)$ encoding time and $\mathcal{O}(n)$ space, we must define a physical memory layout that strictly avoids the $\mathcal{O}(n^2)$ space penalty of dense matrices and the $\mathcal{O}(\text{degree}(v))$ lookup penalty of linked adjacency lists.

**Encoding Phase (Initialization):** During the encoding phase, the algorithm requires three $\mathcal{O}(1)$ lookups per iteration: the child's spatial rank $k$, the leaf's unique remaining neighbor $P$, and the direction bit $d$. All three are resolved without sorting or pointer-chasing by pre-computing three flat arrays in a single $\mathcal{O}(n)$ linear scan over all $n$ vertices.

The $\text{ChildRank}[1 \ldots n+1]$ array assigns to each non-root vertex $v$ its 0-indexed left-to-right position $k$ among its siblings: $\text{ChildRank}[c_k] = k$. The $\text{parent}[1 \ldots n+1]$ array records the directed parent of every non-root vertex, enabling $\mathcal{O}(1)$ resolution of the direction bit $d$. The $\text{nbr}[1 \ldots n+1]$ array accumulates the running sum of a vertex's current neighbor labels; because a leaf by definition has degree 1, its unique remaining neighbor is recovered in $\mathcal{O}(1)$ as $P = \text{nbr}[\text{leaf}]$, with the sum updated in $\mathcal{O}(1)$ at each step via $\text{nbr}[P] \leftarrow \text{nbr}[P] - \text{leaf}$. This neighbor-sum technique eliminates the need for any adjacency list traversal during the main encoding loop.

**Decoding Phase (CSR-Equivalent Flat Allocation):** During the decoding phase (Algorithm 2), the ordered children of each parent $P_i$ must be reconstructed via the spatial map $M$. Allocating individual variable-length arrays for each parent induces severe heap fragmentation and violates strict contiguous $\mathcal{O}(n)$ space constraints.

To resolve this, a Compressed Sparse Row (CSR) equivalent formulation is utilized. During the initial $\mathcal{O}(n)$ sequence scan (Algorithm 2, Lines 4–7), the exact out-degree of every parent is computed. During decoding, the prefix-sum for the BasePointer array must operate purely over the exact out-degree ($\text{out_deg}[v]$) rather than the full decoding degree array $D[v]$. This eliminates $\mathcal{O}(n)$ wasted integer slots, saving approximately 9 MB of memory at the 2.3 million node scale and preserving strict spatial contiguity. This allows the spatial map $M$ to be allocated as a single, contiguous flat array of size $n - 1$. The spatial rank insertion is executed as a strictly $\mathcal{O}(1)$ direct memory offset: $M[\text{BasePointer}[P_i] + k_i] = \text{leaf}$.

## 3.2 Encoding Procedure

To achieve strictly $\mathcal{O}(n)$ time complexity, a sliding-pointer technique is utilized. The degree array $D$ is computed over the augmented unrooted tree $T'$. In $T'$, every non-root vertex $v$ has exactly one parent edge plus any outgoing child edges, giving $D[v] = |\text{children}(v)| + 1$. The root $r$ has no parent edge in the original directed tree, but by Definition 2.3 acquires exactly one additional undirected edge $(r, n+1)$ connecting it to the virtual node; accordingly

$D[r] = |\text{children}(r)| + 1$. The virtual node $n+1$ connects only to $r$, so $D[n+1] = 1$. Crucially, the formula $D[v] = |\text{children}(v)| + 1$ applies uniformly to every vertex including the root: the $+1$ in both cases counts a single undirected connection to the vertex's unique parent-side neighbor in $T'$—for the root, that neighbor is the virtual node $n + 1$; for every other vertex, it is its structural parent. This uniformity is what makes the sliding-pointer loop topology-invariant.

To support arbitrarily deep topologies (e.g., AST depth $> 10{,}000$) and bypass the default 8 MB operating system stack limit, all tree traversals and encoding builders are implemented iteratively using heap-allocated `std::stack` structures.

**Input:** Rooted, directed, ordered tree $T = (V, E, r, <)$ with $|V| = n$

**Output:** Integer sequence $S$ of length $n - 1$

**Algorithm 1** SPPS Encode

```
▷ — Phase 1: Initialization and input validation —
if |E| ≠ n − 1 then reject T
end if                                                  ▷ Necessary condition: not a tree
Initialize ChildRank[1 . . . n + 1] ← 0;   parent[1 . . . n + 1] ← 0;   nbr[1 . . . n + 1] ← 0
for u = 1 to n do ▷ Linear scan; ChildRank is index-independent since each child has a unique parent
    for k = 0 to |children(u)| − 1 do
        Let c be the k-th child of u under <
        ChildRank[c] ← k;   parent[c] ← u
        nbr[u] ← nbr[u] + c;   nbr[c] ← nbr[c] + u   ▷ Accumulate neighbor-sum for O(1) unique-neighbor lookup at leaves
    end for
end for
▷ — Phase 2: Virtual root injection and sliding-pointer encoding —
N ← n + 2
ChildRank[n + 1] ← |children(r)|;   parent[n + 1] ← r   ▷ Virtual node appended as last child of r
nbr[r] ← nbr[r] + (n + 1);   nbr[n + 1] ← r
D[v] ← |children(v)| + 1 for all v ∈ {1, . . . , n};   D[n + 1] ← 1   ▷ Uniform formula: +1 counts the unique parent-side neighbor in T′
Initialize empty sequence S
ptr ← 1
while ptr ≤ n + 1 and D[ptr] ≠ 1 do
    ptr ← ptr + 1
end while
leaf ← ptr
for i = 1 to n do                                       ▷ n − 1 Prüfer steps + 1 virtual-root step
    if D[leaf] == 0 then break
    end if
    P ← nbr[leaf] ▷ When D[leaf] = 1, nbr[leaf] holds exactly the one remaining neighbor
    d ← { +1  if parent[leaf] = P (leaf is a child of P)
        { −1  if parent[P] = leaf (leaf is the parent of P)   ▷ Exactly one case holds for any valid directed tree edge
    k ← ChildRank[leaf]                                 ▷ O(1) lookup
    ω ← d · (P · N + k)                                 ▷ Bijective radix packing
    Append ω to S
    D[P] ← D[P] − 1;   nbr[P] ← nbr[P] − leaf   ▷ Remove leaf from P's neighbor sum
    if D[P] == 1 and P < ptr then   ▷ O(1) shortcut: newly exposed leaf is the current minimum
        leaf ← P
    else
        ptr ← ptr + 1
        while ptr ≤ n + 1 and D[ptr] ≠ 1 do
            ptr ← ptr + 1
        end while
        leaf ← ptr
    end if
end for
Remove the last element of S                            ▷ Discards virtual-root edge; |S| = n − 1
return S
```

### 3.3 Decoding Procedure

During decoding, we bypass spatial sorting of children utilizing $\mathcal{O}(1)$ direct array addressing.

**Input:** Integer sequence $S$ of length $m = n - 1$

**Output:** Reconstructed tree $T = (V, E, r, <)$

**Algorithm 2** SPPS Decode

```
n ← m + 1, N ← n + 2, V ← {1, ..., n + 1}
Initialize degree array D[v] ← 1 for all v ∈ V
Initialize empty edge set E
for each ω_i ∈ S do
    P_i ← ⌊|ω_i|/N⌋
    D[P_i] ← D[P_i] + 1
end for
Let out_deg[v] ← D[v] − 1 for all v ∈ V
Compute prefix-sum BasePointer[v] from
    out_deg[v], allocate single flat array M[0 ... n − 2]
ptr ← 1
while ptr ≤ n + 1 and D[ptr] ≠ 1 do
    ptr ← ptr + 1
end while
leaf ← ptr
for each ω_i ∈ S do
    P_i ← ⌊|ω_i|/N⌋, k_i ← |ω_i| (mod N),
    d_i ← sign(ω_i)                                  ▷ Unpack dimensions via division
    if d_i > 0 then
        Add directed edge (P_i, leaf) to E
    else
        Add directed edge (leaf, P_i) to E
    end if
    M[BasePointer[P_i] + k_i] ← leaf
    D[P_i] ← D[P_i] − 1
    if D[P_i] == 1 and P_i < ptr then
        leaf ← P_i
    else
        ptr ← ptr + 1
        while ptr ≤ n + 1 and D[ptr] ≠ 1 do
            ptr ← ptr + 1
        end while
        leaf ← ptr
    end if
end for
Let {u, v} be the two vertices with D[v] = 1
if u == n + 1 then
    r ← v
else
    r ← u
end if
Remove virtual node n + 1 and its edge from E
return T = (V, E, r) with children of each vertex w
    ordered by M[BasePointer[w]], ..., M[BasePointer[w] + out_deg[w] − 1]
```

## 3.4 Experimental Validation

### 3.4.1 Experimental Environment

All evaluations were conducted on an Apple M1 (ARMv8.5-A architecture) processor. The system utilizes an 8-core asymmetric multiprocessing configuration, featuring 4 Firestorm Performance Cores (P-cores) peaking at 3.2 GHz and 4 Icestorm Efficiency Cores (E-cores) peaking at 2.1 GHz. The memory subsystem consists of 8 GB of Unified LPDDR4X memory with a measured bandwidth of 68.25 GB/s, enforcing a strict 16.00 KiB page size.

The cache hierarchy utilizes a 128-byte cache line size. The L1 Data (L1D) cache provides 128 KB per P-core (64 KB per E-core), while the L1 Instruction (L1I) cache provides 192 KB per P-core (128 KB per E-core). The L2 cache is separated into a 12 MB shared pool for the P-cluster and a 4 MB shared pool for the E-cluster, backed by a 16 MB System-Level Cache (SLC).

Execution occurred on macOS 14.x. Binaries were compiled utilizing Apple clang version 17.0.0 with strict maximum optimization flags (`-std=c++17 -O3`). Baseline evaluations were dynamically linked against Google Protocol Buffers (libprotobuf version 34.0) and Google FlatBuffers (version 25.12.19).

### 3.4.2 Dataset Description

We evaluated topological traversal across two distinct dataset classifications. **Synthetic Datasets** (100,000 and 1,000,000 nodes) were deterministically generated in-memory utilizing `std::mt19937 rng(42)`, simulating AST branching depths via a sliding parent-assignment window in which each node $i$ selects a parent uniformly from the interval $[\max(1, i-500), i-1]$. The **Real Dataset** (2,325,575 nodes) consists of a complete Django framework AST preserving native spatial rank order, ingested sequentially from a flat text file where each line contains a space-separated parent-child directed edge.

### 3.4.3 Benchmark Implementation

Three distinct serialization paradigms were evaluated:

**SPPS (Proposed Method):** Maps the pointer-based tree into a flat integer sequence. Key structures include BasePointer (providing exact offsets into the contiguous memory array $M$), out_deg, and ChildRank (recording sibling order).

**Protocol Buffers (Baseline):** Utilizes a recursive message structure (repeated Node children), which deserializes into a highly fragmented, pointer-heavy object graph layout.

**FlatBuffers (Baseline):** Utilizes a zero-copy schema (table Node with offset vectors) and an internal vtable-based memory layout.

### 3.4.4 Measurement Protocol and Metrics

To mathematically isolate execution from OS-level memory state pollution (e.g., arena allocators, page poisoning), we enforced a rigorous process isolation protocol. Each configuration executed exactly 30 independent trials (TRIALS = 30).

For every measurement cycle, the main process invoked `fork()`, executing the benchmark strictly within the child process. Execution latencies were captured using `std::chrono::high_resolution_clock`, while peak RAM footprint was directly extracted via the OS-reported `rusage.ru_maxrss`.

Two measurement definitions require explicit qualification. **Total Read latency** is defined as the sum of decoding time and DFS traversal time. This captures the full latency to produce a navigable in-memory tree, but the underlying operations differ structurally across formats: SPPS performs an $\mathcal{O}(n)$ decoding pass to reconstruct a heap-allocated pointer

tree, followed by DFS over that tree; FlatBuffers performs no decoding pass and traverses the flat buffer directly via offset arithmetic; Protobuf performs a full message-graph materialization pass, followed by pointer-chasing DFS. These are not identical workloads and comparisons should be interpreted accordingly. **Peak RAM** is captured at the high-water mark of `ru_maxrss` within each trial process, which occurs at different operational phases across formats: for SPPS, this peak includes the input adjacency structure, ChildRank array, degree array, and output integer sequence simultaneously during encoding; for Protobuf, it occurs during the fully materialized decode phase. Absolute peak RAM figures are therefore not directly equivalent, though relative comparisons at the same scale and format reflect real memory pressure differences.

### 3.4.5 Benchmark Results

Table 1: Macro-level Performance at $n = 100{,}000$

| Format | Peak RAM (MB) | Enc (ms) | Dec (ms) | DFS Trav (ms) | Total Read (ms) |
|---|---|---|---|---|---|
| Protobuf | 12.23 ± 0.01 | 11.00 ± 3.46 | 0.12 ± 0.04 | 0.05 ± 0.01 | 0.17 ± 0.05 |
| FlatBuffers | 10.23 ± 0.00 | 12.48 ± 4.06 | 0.00 ± 0.00 | 2.74 ± 0.33 | 2.74 ± 0.33 |
| SPPS (Ours) | 9.98 ± 0.00 | 2.25 ± 0.88 | 1.44 ± 0.42 | 2.47 ± 0.97 | 3.92 ± 1.28 |

Table 2: Macro-level Performance at $n = 1{,}000{,}000$

| Format | Peak RAM (MB) | Enc (ms) | Dec (ms) | DFS Trav (ms) | Total Read (ms) |
|---|---|---|---|---|---|
| Protobuf | 95.35 ± 0.01 | 103.03 ± 12.41 | 0.11 ± 0.01 | 0.04 ± 0.00 | 0.16 ± 0.01 |
| FlatBuffers | 103.59 ± 1.59 | 118.20 ± 17.23 | 0.00 ± 0.00 | 25.33 ± 0.52 | 25.33 ± 0.52 |
| SPPS (Ours) | 77.29 ± 0.02 | 19.63 ± 0.74 | 13.66 ± 4.51 | 22.15 ± 0.79 | 35.81 ± 4.67 |

Table 3: Macro-level Performance at $n = 2{,}325{,}575$ (Django AST)

| Format | Peak RAM (MB) | Enc (ms) | Dec (ms) | DFS Trav (ms) | Total Read (ms) |
|---|---|---|---|---|---|
| Protobuf | 346.83 ± 7.06 | 158.23 ± 12.31 | 94.67 ± 12.89 | 32.22 ± 9.11 | 126.88 ± 19.57 |
| FlatBuffers | 233.18 ± 0.01 | 185.06 ± 5.85 | 0.00 ± 0.00 | 39.83 ± 1.28 | 39.83 ± 1.28 |
| SPPS (Ours) | 172.93 ± 0.01 | 29.61 ± 1.92 | 19.85 ± 1.05 | 25.69 ± 1.13 | 45.54 ± 2.05 |

### 3.4.6 Protocol Buffers Cache-Warmth Effect at Sub-Production Scales

An important observation concerns Protobuf's decode and traversal times at intermediate scales ($n = 100{,}000$ and $n = 1{,}000{,}000$). At these scales, Protobuf reports near-zero DFS traversal times (0.05 ms at $n = 100$K, 0.04 ms at $n = 1$M). This behavior is fully explained by Protobuf's deserialization model: `ParseFromString()` fully materializes the entire tree into a hierarchy of heap-allocated C++ message objects. For trees whose working set fits within or near the CPU's L2 cache (12 MB on Apple M1), the subsequent DFS traversal performs pointer-chasing through memory that is already cache-resident, yielding sub-millisecond traversal times. This is a well-understood cache-warmth artifact, not a general performance advantage.

This phenomenon breaks down at scale. At $n = 2{,}325{,}575$ (Django AST), Protobuf's peak memory footprint reaches 346.83 MB — far exceeding the 12 MB L2 cache. At this scale, Protobuf's decode time rises to 94.67 ms and DFS traversal takes 32.22 ms, for a total read time of 126.88 ms — 2.79× slower than SPPS (45.54 ms). Table 4 summarizes this cache-threshold effect.

Table 4: Protobuf Cache-Threshold Effect Across Scales

| Scale | Dec (ms) | DFS (ms) | Total Read (ms) | Peak RAM (MB) | L2 Status |
|---|---|---|---|---|---|
| 100,000 | 0.12 | 0.05 | 0.17 | 12.23 | Marginal |
| 1,000,000 | 0.11 | 0.04 | 0.16 | 95.35 | Exceeds (data warm) |
| 2,325,575 | 94.67 | 32.22 | 126.88 | 346.83 | Exceeds (cold miss) |

At $n = 1$M, although Protobuf's working set (95 MB) exceeds L2, the decode phase warms the cache for the immediately following DFS traversal. At $n = 2.3$M, the working set is so large (347 MB) that even this warm-up effect is insufficient, and cache misses dominate both phases.

#### 3.4.7 Total Read Performance and FlatBuffers Zero-Copy Advantage

While SPPS achieves 5.3–6.2× faster encoding than both baselines at all scales, total read performance (decode + DFS traversal) presents a more nuanced picture. FlatBuffers employs a zero-copy deserialization model where no parsing step is required — the serialized buffer is accessed directly through offset-based pointer arithmetic. This gives FlatBuffers a structural advantage in read-path performance, as shown in Table 5.

Table 5: Total Read Latency (Decode + DFS Traversal) Across Scales

| Scale | SPPS (ms) | FlatBuffers (ms) | Protobuf (ms) | Read Winner |
|---|---|---|---|---|
| 100,000 | 3.92 | 2.74 | 0.17* | Protobuf* |
| 1,000,000 | 35.81 | 25.33 | 0.16* | Protobuf* |
| 2,325,575 | 45.54 | 39.83 | 126.88 | FlatBuffers |

*Protobuf's anomalously fast reads at 100K–1M are a cache-warmth artifact (Section 3.4.6); this advantage disappears at production scale.

At the production-relevant scale of $n = 2{,}325{,}575$, FlatBuffers achieves 39.83 ms total read versus SPPS's 45.54 ms — a 1.14× advantage for read-path latency. However, this must be weighed against SPPS's substantial advantages in the write path and memory footprint: SPPS encodes 6.25× faster than FlatBuffers (29.61 ms vs. 185.06 ms), consumes 1.35× less peak RAM (172.93 MB vs. 233.18 MB), and produces a 2.5× smaller serialized payload (18.60 MB vs. 46.51 MB at $n = 2.3$M) with no schema definition required.

For write-heavy workloads (e.g., AST serialization during compilation, NAS checkpoint snapshotting), SPPS's encoding throughput advantage dominates. For read-heavy workloads with infrequent writes, FlatBuffers' zero-copy model provides faster random access, though at significantly higher serialization cost and memory overhead.

#### 3.4.8 Scalability and Memory Layout Analysis

Empirical bounds confirm the strict $\mathcal{O}(n)$ spatial and temporal complexity of the proposed algorithm. Across a scale variation spanning more than two orders of magnitude from 100,000 to 2.3 million nodes, SPPS encoding latency remains strictly bounded within a narrow range of 12.73 to 22.50 ns/node, empirically validating the linear $\mathcal{O}(n)$ behavior claimed in Theorem 2.2. At maximum scale ($n = 2{,}325{,}575$), SPPS encoding processes 78.54 MNodes/s, yielding a 5.3× speedup over Protobuf (14.70 MNodes/s) and a 6.2× speedup over FlatBuffers (12.57 MNodes/s).

Furthermore, during the deserialization phase at maximum scale, SPPS achieves a sustained ingestion bandwidth of 937.1 MB/s (where MB denotes $10^6$ bytes, i.e., SI megabytes; equivalently 893.7 MiB/s using binary mebibytes), representing a 6.2× throughput advantage over the Protocol Buffers parsing engine (150.5 MB/s); FlatBuffers is excluded from

this specific decode bandwidth comparison as it utilizes instantaneous zero-copy memory mapping.

Spatial complexity is similarly linear and constrained. At 2.3 million nodes, Protobuf fragments to approximately 156.4 bytes/node (346.83 MB peak RAM). SPPS maintains a strictly bounded layout of approximately 78.0 bytes/node (172.93 MB peak RAM), effectively halving the spatial footprint. Furthermore, regarding serialized payload determinism, SPPS enforces a fixed-width 8.00 bytes/node wire format across all scales, trading wire compactness against variable-length formats for constant-time decoding and hardware-aligned cache access. In contrast, Protobuf's varint compression fluctuates between 5.12 and 6.17 bytes/node, and FlatBuffers suffers from a rigid 20.00 bytes/node vtable overhead.

Table 6: Theoretical and Spatial Complexity Comparison

| Metric | SPPS (Proposed) | Protocol Buffers | FlatBuffers |
|---|---|---|---|
| Encode Time | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ amortized | $\mathcal{O}(n)$ amortized |
| Decode Time | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ zero-copy |
| DFS Traversal | $\mathcal{O}(n)$ contiguous | $\mathcal{O}(n)$ pointer-chase | $\mathcal{O}(n)$ offset-chase |
| Serialized Payload | Exactly $8n$ bytes | $\sim 6n$ bytes (varint) | $\sim 20n$ bytes (fixed) |
| Schema Required | No | Yes (.proto) | Yes (.fbs) |

As summarized in Table 6, SPPS is the only format that provides strictly contiguous $\mathcal{O}(n)$ traversal without requiring external schema definitions or suffering from pointer-chasing memory fragmentation.

### 3.4.9 Serialized Size Analysis

SPPS achieves a constant serialization cost of exactly 8.00 bytes per node across all tree sizes. This follows directly from the encoding: the output is a sequence of $(n-1)$ signed 64-bit integers, yielding exactly $8(n-1)$ bytes regardless of tree topology or node label distribution — a unique determinism among the evaluated methods.

FlatBuffers uses a fixed-overhead format with vtables and offset vectors, resulting in a constant 20.00 bytes per node regardless of tree structure — 2.5× larger than SPPS. Protobuf's bytes-per-node varies from 5.12 (at $n = 100$K) to 6.17 (at $n = 1$M) due to its variable-length integer (varint) encoding: as tree size grows, average node IDs increase, requiring more varint bytes per field. Despite Protobuf's tighter wire format at intermediate scales, its deserialization requires full parsing and heap allocation of message objects, contributing to its 2.01× higher peak RAM at $n = 2.3$M.

Table 7: Serialized Wire Format (bytes/node)

| Scale | SPPS | FlatBuffers | Protobuf |
|---|---|---|---|
| 100,000 | 8.0000 | 20.0001 | 6.0135 |
| 1,000,000 | 8.0000 | 20.0000 | 6.1743 |
| 2,325,575 | 8.0000 | 20.0000 | 6.1260 |

### 3.4.10 Memory Scaling Analysis

Table 8 reports peak RAM per node across scales, revealing whether overhead grows linearly or superlinearly.

Table 8: Peak RAM Per Node (bytes/node)

| Scale | SPPS | FlatBuffers | Protobuf |
|---|---|---|---|
| 100,000 | 104.7 | 107.3 | 128.2 |
| 1,000,000 | 81.1 | 108.6 | 100.0 |
| 2,325,575 | 78.0 | 105.1 | 156.4 |

SPPS's per-node memory cost decreases from 104.7 bytes/node at $n = 100$K to 78.0 bytes/node at $n = 2.3$M, reflecting the amortization of fixed overhead (runtime stack, vector metadata, OS page tables) across more nodes. The asymptotic cost stabilizes near 78 bytes/node, confirming $\mathcal{O}(n)$ spatial complexity.

Protobuf's per-node cost shows non-monotonic behavior: it decreases from 128.2 bytes/node at $n = 100$K to 100.0 bytes/node at $n = 1$M, then rises sharply to 156.4 bytes/node at $n = 2.3$M. The initial decrease reflects amortization of fixed process and library overhead across more nodes. The sharp rise at $n = 2.3$M is caused by heap fragmentation at scale: each C++ message object carries vtable pointer, arena metadata, and repeated-field vector overhead, and at millions of nodes these allocations fragment the heap into non-contiguous regions, increasing effective per-node overhead. FlatBuffers maintains a relatively stable ∼105–109 bytes/node, consistent with its pre-allocated buffer model but 1.35× higher than SPPS due to vtable and offset vector overhead per node.

#### 3.4.11 Per-Node Encoding Cost and $\mathcal{O}(n)$ Verification

Table 9 reports per-node encoding latency in nanoseconds. A strictly $\mathcal{O}(n)$ algorithm should exhibit approximately constant ns/node regardless of input size.

Table 9: Per-Node Encoding Latency (ns/node)

| Scale | SPPS | FlatBuffers | Protobuf |
|---|---|---|---|
| 100,000 | 22.5 | 124.8 | 110.0 |
| 1,000,000 | 19.6 | 118.2 | 103.0 |
| 2,325,575 | 12.7 | 79.6 | 68.0 |

SPPS encoding cost remains in the narrow range of 12.7–22.5 ns/node across the full evaluated scale, empirically confirming $\mathcal{O}(n)$ time complexity. The slight decrease at $n = 2.3$M (12.7 ns/node) is consistent with improved instruction cache utilization for the tight encoding loop at larger working sets, where the loop body remains fully resident in L1 I-cache while data streams from DRAM in a predictable sequential pattern. FlatBuffers and Protobuf show 79–125 ns/node encoding cost — 4–6× slower than SPPS — due to dynamic memory allocation overhead (arena allocators, vtable construction, and offset vector management).

#### 3.4.12 Cache, Hardware, and OS Observations

Hardware tracing was conducted utilizing the Apple M1 Performance Monitoring Unit (PMU) sampled at 1000 Hz via xctrace. Analysis of pipeline execution reveals that when the 2.3 million node dataset exceeds the 12 MB L2 cache capacity, traditional recursive traversals (e.g., Protocol Buffers and FlatBuffers) trigger severe instruction delivery bottlenecks. Peak pipeline stall rates observed during these traversals exceeded 50%, and peak discarded execution slots due to branch mispredictions similarly exceeded 50%, consistent with the pointer-chasing and indirect-branch behavior characteristic of these formats at this scale.

Over the combined 5.89-second profiling run (all methods, all scales), aggregate pipeline utilization breaks down as follows: 56.42% of pipeline slots retired useful instructions, 9.11%

were lost to instruction delivery stalls, 19.39% were discarded due to branch mispredictions, and 15.09% stalled on execution dependencies. The 19% aggregate misprediction rate originates predominantly from Protobuf and FlatBuffers' recursive and pointer-chasing code paths. In contrast, SPPS's sequential array access pattern achieves streaming prefetch behavior, keeping instruction delivery stalls below 5% during its encoding and decoding phases.

By completely flattening the topology into contiguous memory arrays, SPPS mitigates these micro-architectural penalties, achieving a remarkably stable traversal Coefficient of Variation (CV%) of 4.40%, in contrast to the cache-sensitive 28.28% CV% exhibited by Protocol Buffers.

**Tail Latency Analysis.** For latency-sensitive applications (real-time compilers, game engines, interactive tools), worst-case performance is often more important than average performance. Table 10 and Table 11 report percentile encoding and total read latency at the production scale ($n = 2{,}325{,}575$).

Table 10: Encoding Latency Percentiles at $n = 2{,}325{,}575$ (ms)

| Method | P50 | P90 | P95 | P99 | Max | P99−P50 |
|---|---|---|---|---|---|---|
| SPPS | 29.10 | 31.03 | 34.54 | 35.43 | 35.54 | 6.33 |
| FlatBuffers | 182.70 | 189.80 | 192.38 | 205.74 | 210.46 | 23.04 |
| Protobuf | 152.57 | 171.22 | 188.75 | 195.98 | 196.81 | 43.41 |

Percentiles computed via linear interpolation over 30 trials (numpy convention).
With $n = 30$, P99 interpolates between the 29th and 30th observations; values above P95 should be interpreted as indicative rather than statistically stable estimates.

Table 11: Total Read Latency Percentiles at $n = 2{,}325{,}575$ (ms)

| Method | P50 | P90 | P95 | P99 | Max | P99−P50 |
|---|---|---|---|---|---|---|
| SPPS | 44.84 | 47.06 | 49.44 | 52.92 | 53.70 | 8.08 |
| FlatBuffers | 39.14 | 41.67 | 42.49 | 42.69 | 42.75 | 3.55 |
| Protobuf | 121.09 | 140.39 | 165.37 | 198.99 | 207.05 | 77.89 |

Percentiles computed via linear interpolation over 30 trials (numpy convention).
Values above P95 should be interpreted as indicative with $n = 30$ samples.

SPPS demonstrates predictable tail behavior: the P99 encoding latency is only 1.22× the median, and the P99−P50 gap of 6.33 ms represents just 21.8% of the median encoding time. In contrast, Protobuf's P99−P50 gap for total read time is 77.89 ms (64.3% of median), indicating severe tail-latency unpredictability driven by variable cache miss patterns during pointer-chasing DFS through a 347 MB message tree. FlatBuffers achieves the tightest read tail distribution (P99−P50 = 3.55 ms) owing to its deterministic offset-based access pattern, but at the cost of extreme encoding tail latency (P99−P50 = 23.04 ms). For systems requiring bounded worst-case latency, SPPS provides the most balanced profile: tight encoding tails and acceptable read tails.

At the operating system level, execution tracing logged tens of thousands of virtual memory interventions—including copy-on-write (COW) faults, page cache hits, and memory decompression events—across the combined profiling run. This OS-level activity validates the necessity of the isolated `fork()` benchmarking protocol, ensuring metrics remain insulated from transient system perturbations.

### 3.5 Limitations, Reproducibility, and Validity

While SPPS provides superior archival contiguity, it strictly supports tree structures and remains vulnerable to deep recursion stack limits on degenerate graphs. Furthermore, preventing composite integer overflow logically caps the maximum tree size at $n \leq \lfloor 2^{31.5} \rfloor$ nodes

on 64-bit architectures. It also exhibits decoding temporal overhead compared to memory-mapped flat buffers.

To ensure statistical validity, all reported metrics represent the mean across 30 process-isolated trials, with standard deviations provided to transparently bound temporal and spatial variance. To support independent verification, the complete implementation, build instructions (CMake/Makefile), and dataset generation scripts are available via our public repository at https://github.com/YOUR-USERNAME/YOUR-REPO.

### 3.5.1 Threats to Validity

**Internal Validity.** All measurements use `fork()`-based process isolation, ensuring that each of the 360 timed trials runs in a clean OS process with independent memory state. Means and standard deviations are reported across 30 trials per configuration. However, as with any systems benchmark, background OS activity (kernel scheduling, memory compression, and I/O interrupts) can introduce unpredictable variance. The `fork()`-based isolation partially mitigates cold-start effects but does not eliminate all sources of thermal throttling or CPU frequency scaling on the Apple M1 platform.

**External Validity.** Our experiments are conducted on a single hardware platform (Apple M1, 8-core ARM64, 8 GB unified memory). While the relative performance characteristics — cache-line-sequential access versus pointer-chasing — are architecture-general, absolute timings may differ on x86-64 platforms with different cache hierarchies, memory bandwidths, or branch predictor designs. We evaluate on two synthetic AST-like trees ($n = 100$K, $n = 1$M) generated with bounded parent distance (parent $\in [\max(1, i - 500),\ i - 1]$, seed $= 42$), which produces moderately branching trees but may not represent all topologies encountered in practice (e.g., highly skewed path graphs or perfectly balanced trees). We include one real-world dataset (Django AST, $n = 2{,}325{,}575$) to validate results on production data; additional datasets (e.g., LLVM IR trees, XML document trees, file system hierarchies) would further strengthen generalizability.

**Construct Validity.** We measure serialized size in bytes, capturing the on-wire cost but not accounting for potential gains from downstream general-purpose compression (e.g., zstd, LZ4). Protobuf's varint encoding achieves 5.12–6.17 bytes/node, which is smaller than SPPS's fixed 8.00 bytes/node on the wire. However, SPPS's integer sequences may be highly compressible due to their arithmetic structure (correlated parent labels $P_i$ for locally ordered trees), and this dimension is left for future investigation.

## 4 Conclusion

We introduced the Signed Positional Prüfer Sequence (SPPS), a pure integer-array encoding for rooted, directed, and ordered trees that eliminates the need for external metadata arrays. By combining virtual root injection with bijective radix packing, SPPS achieves lossless reconstruction that preserves topology, root, edge direction, and spatial ordering through a pure bijection without auxiliary state structures. At scale, it reduces peak memory fragmentation by 50%, operating at a fixed-width 8.00 bytes/node (172.93 MB at 2.3 million nodes) and maintaining strict $\mathcal{O}(n)$ space complexity. In terms of throughput, SPPS operates in strictly $\mathcal{O}(n)$ linear time via sliding-pointer logic, achieving encoding rates of 78.54 MNodes/s and yielding a 5.3× speedup over Google Protocol Buffers. Finally, by restructuring serialization into cache-sequential integer arrays and substantially reducing pointer-chasing during encoding, traversal latency variance is bounded to 4.40%, providing a highly contiguous integer-array format well suited for high-performance computing environments.

Future work will explore several directions. **Variable-Radix SPPS Variants** would scale the radix by the local parent degree rather than a global constant $N$, minimizing bit-

waste for skewed trees. **Compression of SPPS Sequences** is a promising avenue: consecutive sequence elements often have correlated parent labels $P_i$ for trees with spatial locality (such as ASTs), and delta encoding followed by variable-length integer compression (e.g., varint, Simple8b) or general-purpose compression (zstd, LZ4) could reduce the wire format below Protobuf's 5–6 bytes/node while preserving the bijective property and $\mathcal{O}(n)$ complexity. **Streaming Encoding** — an online variant that emits sequence elements as tree nodes arrive (e.g., during SAX-style parsing) — would extend SPPS's applicability to memory-constrained environments. Finally, comprehensive empirical benchmarking against modern zero-copy frameworks like Cap'n Proto for serializing distributed ML topologies (ONNX) and compiler intermediate representations (LLVM Bitcode), as well as evaluation on diverse real-world tree corpora (LLVM IR, OpenStreetMap XML, phylogenetic databases), constitute important directions for establishing broader external validity.

**Acknowledgments:** The authors thank the College of Engineering Munnar and APJ Abdul Kalam Technological University for institutional support.

## References

[1] H. Prüfer, "Neuer Beweis eines Satzes über Permutationen," *Archiv der Mathematik und Physik*, vol. 27, pp. 142–144, 1918.

[2] A. Cayley, "A theorem on trees," *The Quarterly Journal of Pure and Applied Mathematics*, vol. 23, pp. 376–378, 1889.

[3] E. H. Neville, "The codifying of tree-structure," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 49, no. 3, pp. 381–385, 1953.

[4] J. W. Moon, *Counting Labelled Trees*, Canadian Mathematical Congress, 1970.

[5] P. Rao and B. Moon, "PRIX: Indexing and querying XML using Prüfer sequences," in *Proceedings of the 20th International Conference on Data Engineering (ICDE)*, pp. 288–300, 2004.

[6] J. Kwon, P. Rao, B. Moon, and S. Lee, "FIST: Scalable XML document filtering by sequencing twig patterns," in *Proceedings of the 31st International Conference on Very Large Data Bases (VLDB)*, pp. 217–228, 2005.

[7] G. Jacobson, "Space-efficient static trees and graphs," in *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 549–554, 1989.

[8] J. I. Munro and V. Raman, "Succinct representation of balanced parentheses and static trees," *SIAM Journal on Computing*, vol. 31, no. 3, pp. 762–776, 2001.

[9] A. Darling and D. Fickes, "Prüfer codes on vertex-colored rooted trees," arXiv preprint arXiv:2506.15796, June 2025.

[10] J. Fischer, D. Köppl, and K. Sadakane, "Hypersuccinct trees—new universal tree source codes for optimal compressed tree data structures," arXiv preprint arXiv:2104.13457, April 2021.

[11] F. Kurpicz, "Engineering compact data structures for rank and select queries," in *Proceedings of the 29th International Symposium on String Processing and Information Retrieval (SPIRE)*, pp. 257–272, 2022.

[12] S. Klausa and E. Sen, "Exceptional sequences and rooted labeled forests," *Advances in Mathematics*, vol. 456, article 109891, 2024.